

Name: D. Bala Charan

ID number: 2520090087

Practical Week 7

Create a User — POST

```
Windows PowerShell
-----
dar--l      21-09-2026   14:48      controllers
dar--l      21-09-2026   14:48      node_modules
dar--l      21-09-2026   14:48      routes
-a---l      21-09-2026   14:44      39264 package-lock.
                                     json
-a-----   21-09-2026   14:44      349 package.json

PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\Week 7\n
odememo\user-service> node app.js
User service running on port 5001
MongoDB connected
```

```
Windows PowerShell
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\Week 7\n
odememo\user-service> Invoke-WebRequest -Uri "http://localhost:5
001/users" -Method POST -ContentType "application/json" -Body '{
"username": "Charan", "email": "2520090087@klh.edu.in", "profile":
{} }'

message      user
-----
User created successfully @{username=Charan; email=2520090...
```

POST in mongosh:

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:5001/users`
- Method:** `POST`
- Body (raw):**

```
{
  "username": "Charan",
  "email": "2520090087@klh.edu.in"
}
```
- Response (JSON):**

```
{
  "message": "User created successfully",
  "user": {
    "username": "Charan",
    "email": "2520090087@klh.edu.in",
    "_id": "6ab0f91bf620804aca5e8a07",
    "__v": 0
  }
}
```
- Status:** 201 Created
- Time:** 6 ms
- Size:** 386 B

Get User — GET

DUDDUKURI's Workspace

Overview GET http://localhost:5001/u...

http://localhost:5001/users/6ab0f91bf620804aca5e8a07

GET http://localhost:5001/users/6ab0f91bf620804aca5e8a07

Send

Body Cookies Headers 6 Test Results

200 OK · 4 ms · 333 B

JSON Preview Visualize

```
1 {
2   "_id": "6ab0f91bf620804aca5e8a07",
3   "username": "Charan",
4   "email": "2520090087@klh.edu.in",
5   "__v": 0
6 }
```

Output on the browser:

localhost:5001/users/6ab0f91bf620004aca5e8a07

Pretty print

```
{
  "_id": "6ab0f91bf620004aca5e8a07",
  "username": "Charan",
  "email": "2520090087@klh.edu.in",
  "__v": 0
}
```

Create Activity — POST

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\Week 7\nodememo\user-service> Invoke-RestMethod -Uri "http://localhost:5001/activities" -Method POST -ContentType "application/json" -Body '{"user_id":"6ab0f91bf620004aca5e8a07","action":"SEARCH_PYTHON"}'
```

message activity

Activity created successfully @user_id=6ab0f91bf620004aca5e...

DUDDUKURI's Workspace

Overview POST http://localhost:5001/...

http://localhost:5001/activities

POST http://localhost:5001/activities

Send

Body Cookies Headers 8 Test Results

201 Created · 9 ms · 439 B

JSON Preview Visualize

```
1 {
2   "message": "Activity created successfully",
3   "activity": {
4     "user_id": "6ab0f91bf620004aca5e8a07",
5     "action": "SEARCH_JAVA",
6     "_id": "6ab0f9e1620084aca5e8a08",
7     "timestamp": "2026-09-21T09:40:46.119Z",
8     "__v": 0
9   }
10 }
```

Add More Activities

DUDDUKURI's Workspace

Overview POST http://localhost:5001/activities/6ab0f91bf620004aca5e8a07

POST http://localhost:5001/activities/6ab0f91bf620004aca5e8a07

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary

```
1 {
2   "user_id": "6ab0f91bf620004aca5e8a07",
3   "activities": [
4     {
5       "action": "Perform Week 7 Skill"
6     },
7     {
8       "action": "Perform Week 7 Practical"
9     },
10    {
11      "action": "Complete MongoDB Task"
12    },
13    {
14      "action": "Test APIs in Postman"
15    }
16  ]
17 }
```

Body Cookies Headers 7 Test Results

{ } JSON Preview Visualize

```
1 [
2   {
3     "_id": "6ab10015f620004aca5e8a0c",
4     "user_id": "6ab0f91bf620004aca5e8a07",
5     "action": "SEARCH_PYTHON",
6     "timestamp": "2026-09-21T09:59:49.270Z",
7     "_v": 0
8   },
9   {
10    "_id": "6ab0fe44f620004aca5e8a09",
11    "user_id": "6ab0f91bf620004aca5e8a07",
12    "action": "Perform Week 7 Practical",
13    "timestamp": "2026-09-21T09:52:04.478Z",
14    "_v": 0
15  }
16 ]
```

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\Week 7\nodememo\user-service> Invoke-WebRequest -Uri "http://localhost:5001/activities/6ab0f91bf620004aca5e8a07" -Method GET

_id      : 6ab10015f620004aca5e8a0c
user_id  : 6ab0f91bf620004aca5e8a07
action   : SEARCH_PYTHON
timestamp : 2026-09-21T09:59:49.270Z
_v       : 0

_id      : 6ab0fe44f620004aca5e8a09
user_id  : 6ab0f91bf620004aca5e8a07
action   : Perform Week 7 Practical
timestamp : 2026-09-21T09:52:04.478Z
_v       : 0

_id      : 6ab0fb9ef620004aca5e8a08
user_id  : 6ab0f91bf620004aca5e8a07
action   : SEARCH_JAVA
timestamp : 2026-09-21T09:40:46.119Z
_v       : 0
```

Output on the browser:

< > ↺

localhost:5001/activities/6ab0f91bf620004aca5e8a07

Pretty print ☒

```
[
  {
    "_id": "6ab10015f620004aca5e8a0c",
    "user_id": "6ab0f91bf620004aca5e8a07",
    "action": "SEARCH_PYTHON",
    "timestamp": "2026-09-21T09:59:49.270Z",
    "_v": 0
  },
  {
    "_id": "6ab0fe44f620004aca5e8a09",
    "user_id": "6ab0f91bf620004aca5e8a07",
    "action": "Perform Week 7 Practical",
    "timestamp": "2026-09-21T09:52:04.478Z",
    "_v": 0
  },
  {
    "_id": "6ab0fb9ef620004aca5e8a08",
    "user_id": "6ab0f91bf620004aca5e8a07",
    "action": "SEARCH_JAVA",
    "timestamp": "2026-09-21T09:40:46.119Z",
    "_v": 0
  }
]
```

MongoDB connection:

MongoDB Compass - 127.0.0.1:27017/digital_library.activities

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (3)

Search connections

127.0.0.1:27017

- BookFlow
- College
- admin
- config
- digital_library
 - activities
 - users
- local
- student_records
- Dbms1
- Week-3_Practical

127.0.0.1:27017 > digital_library > activities

Documents 3 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

25 1 - 3 of 3

```
{ "_id": ObjectId("6ab0fb9ef620004aca5e8a00"),
  "user_id": ObjectId("6ab0f91bf620004aca5e8a07"),
  "action": "SEARCH_JAVA",
  "timestamp": ISODate("2026-09-21T09:40:46.119+00:00") }
21 September 2026 at 09:40:46 UTC
__v: 0
```

```
{ "_id": ObjectId("6ab0fe44f620004aca5e8a09"),
  "user_id": ObjectId("6ab0f91bf620004aca5e8a07"),
  "action": "Perform Week 7 Practical",
  "timestamp": ISODate("2026-09-21T09:52:04.478+00:00") }
21 September 2026 at 09:52:04 UTC
__v: 0
```

```
{ "_id": ObjectId("6ab10015f620004aca5e8a0c"),
  "user_id": ObjectId("6ab0f91bf620004aca5e8a07"),
  "action": "SEARCH_PYTHON",
  "timestamp": ISODate("2026-09-21T09:59:49.270+00:00") }
21 September 2026 at 09:59:49 UTC
__v: 0
```

MongoDB Compass - 127.0.0.1:27017/digital_library.users

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (3)

Search connections

127.0.0.1:27017

- BookFlow
- College
- admin
- config
- digital_library
 - activities
 - users
- local
- student_records
- Dbms1
- Week-3_Practical

127.0.0.1:27017 > digital_library > users

Documents 4 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain

25 1 - 4 of 4

```
{ "_id": ObjectId("6ab0f735f620004aca5e8a05"),
  "username": "Divya",
  "email": "divya@gmail.com" }
__v: 0
```

```
{ "_id": ObjectId("6ab0f7fef620004aca5e8a06"),
  "username": "Priyanka",
  "email": "2520090001@klh.edu.in" }
__v: 0
```

```
{ "_id": ObjectId("6ab0f91bf620004aca5e8a07"),
  "username": "Priyanka",
  "email": "2520090001@klh.edu.in" }
__v: 0
```

```
{ "_id": ObjectId("6ab10324f620004aca5e8a0e"),
  "username": "Priyanka",
  "email": "2520090001@klh.edu.in" }
__v: 0
```

db.js

```
const mongoose = require("mongoose");
const MONGO_URI =
  process.env.MONGO_URI ||
  "mongodb://127.0.0.1:27017/digital_library";
mongoose.connect(MONGO_URI)
  .then(() => console.log("MongoDB connected"))
  .catch(err => console.error("MongoDB connection error:", err));
const userSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true
  },
  profile: {
    type: mongoose.Schema.Types.Mixed,
    default: {}
  }
});
const activitySchema = new mongoose.Schema({
  user_id: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: "User"
  },
  action: {
    type: String,
    required: true
  },
  timestamp: {
    type: Date,
    default: Date.now
  }
});
const User = mongoose.model("User", userSchema);
const Activity = mongoose.model("Activity", activitySchema);
module.exports = { User, Activity };
```

controllers/userController.js

```
const { User } = require("../db");
const createUser = async (req, res) => {
  try {
    const { username, email, profile } = req.body;
    const user = await User.create({
      username,
      email,
      profile
    });
    res.status(201).json({
      message: "User created successfully",
      user
    });
  } catch (error) {
    res.status(500).json({
      message: "Error creating user",
      error: error.message
    });
  }
};

const getUser = async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user)
      return res.status(404).json({
        message: "User not found"
      });
    res.json(user);
  } catch (error) {
    res.status(500).json({
      message: "Error getting user",
      error: error.message
    });
  }
};

module.exports = { createUser, getUser };
```

controllers/activityController.js

```
const mongoose = require("mongoose");
const { Activity } = require("../db");
const createActivity = async (req, res) => {
  try {
    const { user_id, action } = req.body;
    if (!mongoose.Types.ObjectId.isValid(user_id))
      return res.status(400).json({
        message: "Invalid user_id"
      });
    const activity = await Activity.create({
      user_id,
      action
    });
    res.status(201).json({
      message: "Activity created successfully",
      activity
    });
  } catch (error) {
    res.status(500).json({
      message: "Error creating activity",
      error: error.message
    });
  }
};

const getActivities = async (req, res) => {
  try {
    const { user_id } = req.params;
    if (!mongoose.Types.ObjectId.isValid(user_id))
      return res.status(400).json({
        message: "Invalid user_id"
      });
    const activities = await Activity
      .find({ user_id })
      .sort({ timestamp: -1 });
    res.json(activities);
  } catch (error) {
    res.status(500).json({
      message: "Error getting activities",
      error: error.message
    });
  }
};

module.exports = { createActivity, getActivities };
```

routes/userRoutes.js

```
const express = require("express");
const router = express.Router();
const {
  createUser,
  getUser
} = require("../controllers/userController");
```

```
router.post("/users", createUser);
router.get("/users/:id", getUser);
module.exports = router;
```

routes/activityRoutes.js

```
const express = require("express");
const router = express.Router();
const {
  createActivity,
  getActivities
} = require("../controllers/activityController");
router.post("/activities", createActivity);
router.get("/activities/:user_id", getActivities);
module.exports = router;
```

app.js

```
const express = require("express");
const app = express();
require("./db");
app.use(express.json());
const activityRoutes = require("./routes/activityRoutes");
const userRoutes = require("./routes/userRoutes");
app.use("/", activityRoutes);
app.use("/", userRoutes);
app.listen(5001, () => {
  console.log("User service running on port 5001");
});
```